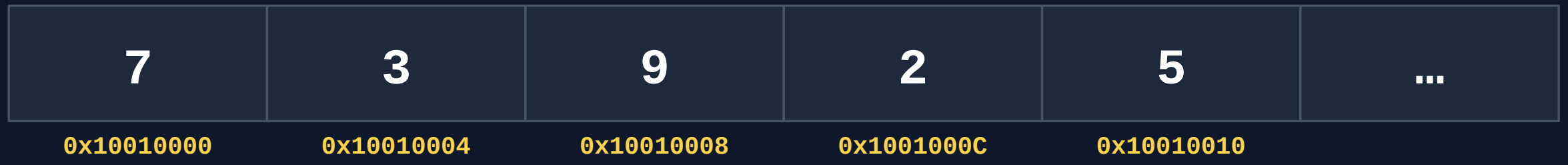
ENCONTRO 3 DE 6

Memória

Curso Introdutório de Programação de Baixo Nível

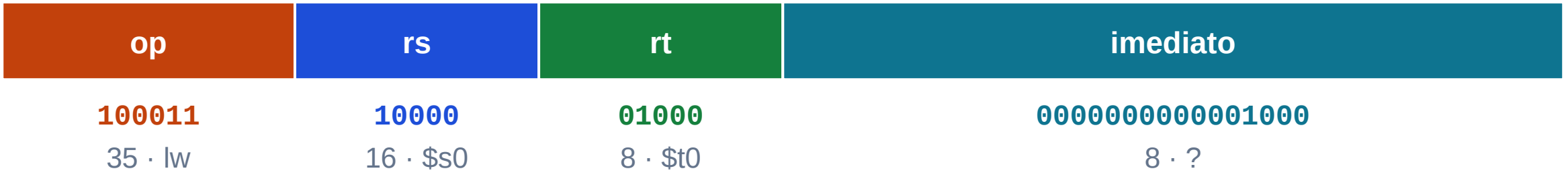


1 · DE ONDE PARAMOS

Semana passada: 32 registradores é pouco

O encontro 2 terminou com uma instrução que vocês já sabiam decompor, mas não sabiam ler:

```
lw  $t0, 8($s0)      →      0x8E080008
```

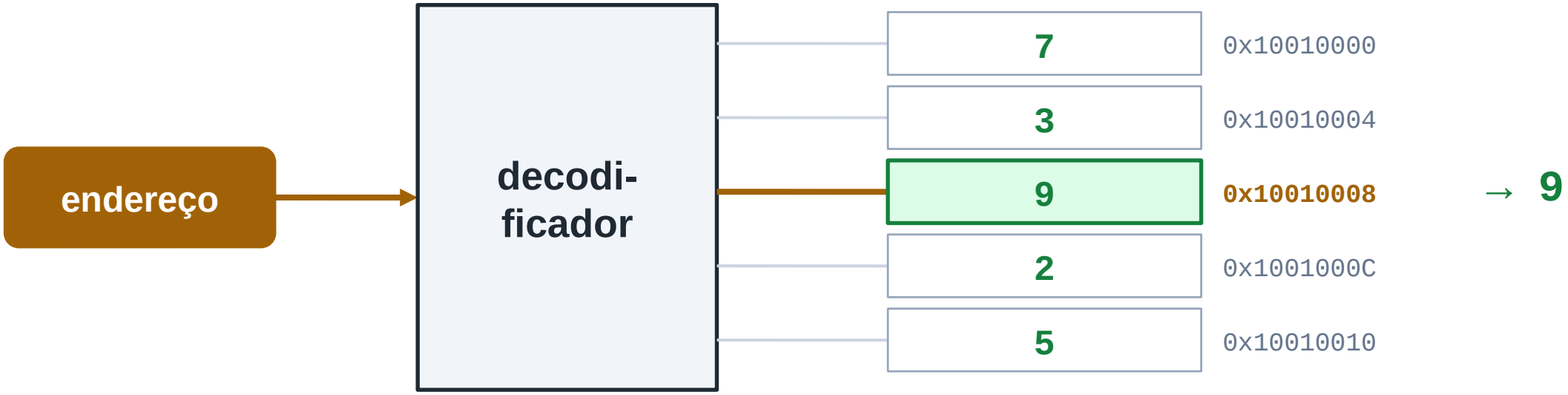


Três perguntas para hoje:

- O que é um endereço?
- O que o $s0 tem dentro — e por que ele está entre parênteses?
- Por que 8, e não 2, para chegar ao terceiro número?

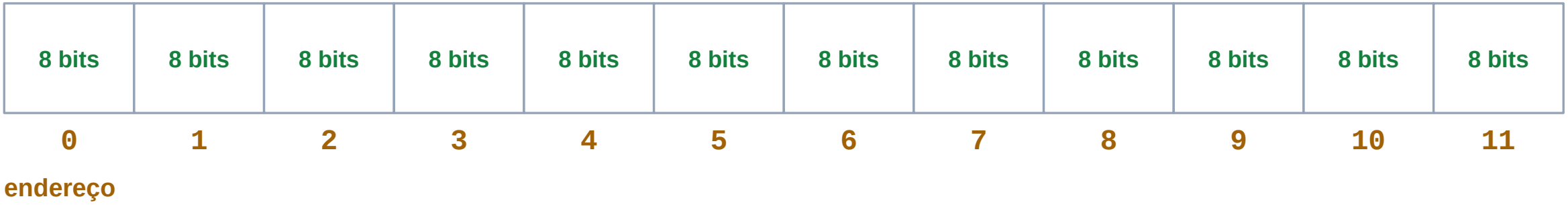
Bancada e almoxarifado: hoje vamos até o almoxarifado buscar coisas — e devolver.

O decodificador escolhe qual gaveta da memória acorda



Na disciplina: um decodificador de n entradas ativa exatamente uma de 2^n saídas. **Aqui:** as entradas são os bits do endereço; a saída ativa é a palavra que vai ser lida ou escrita.

Uma fila enorme de bytes, cada um com um número



Endereço

O número da posição. Não muda nunca: a posição 8 é sempre a posição 8.

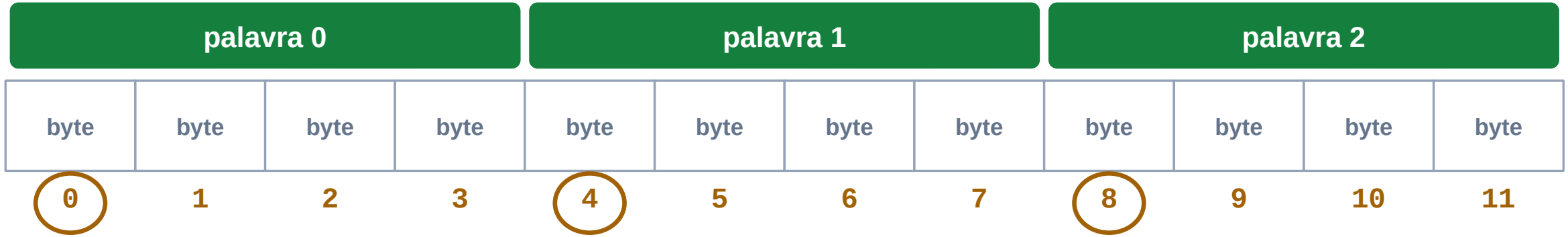
Conteúdo

O que está guardado ali: 1 byte = 8 bits. Muda quando alguém escreve.

Tamanho

Endereços têm 32 bits: até $2^{32} \approx$ 4 bilhões de posições.

Uma palavra = 4 bytes = 32 bits



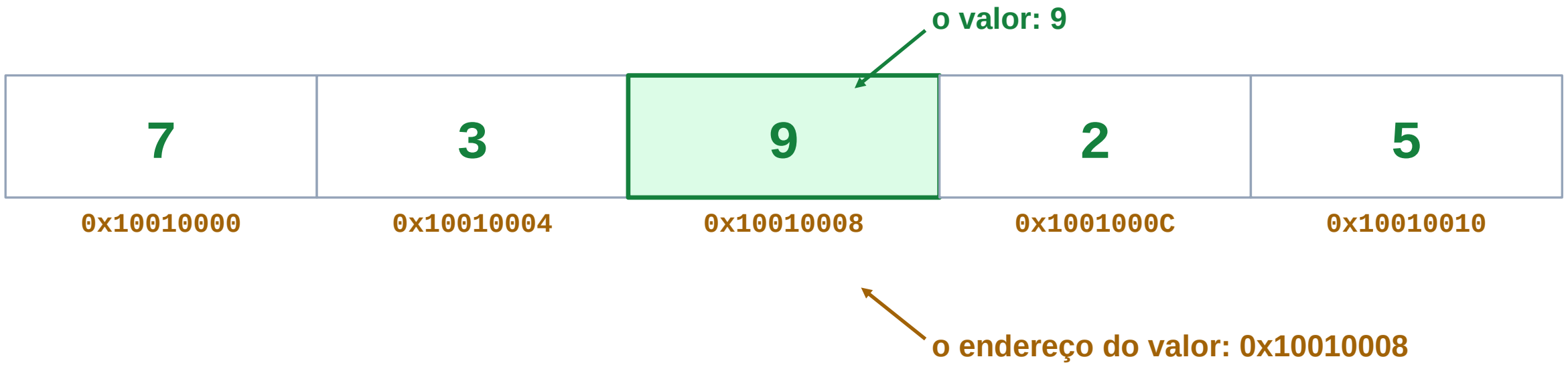
Palavras começam em múltiplos de 4: 0, 4, 8, 12… Por isso os endereços do vetor pulam de 4 em 4 — o mesmo motivo de as instruções pularem de 4 em 4 desde o encontro 1.

Alinhamento

Ler uma palavra a partir do endereço 6 — no meio de duas — o MIPS não faz. O MARS para o programa:
`address not aligned on word boundary`

5 · A DISTINÇÃO MAIS IMPORTANTE DA AULA

O valor e o endereço do valor

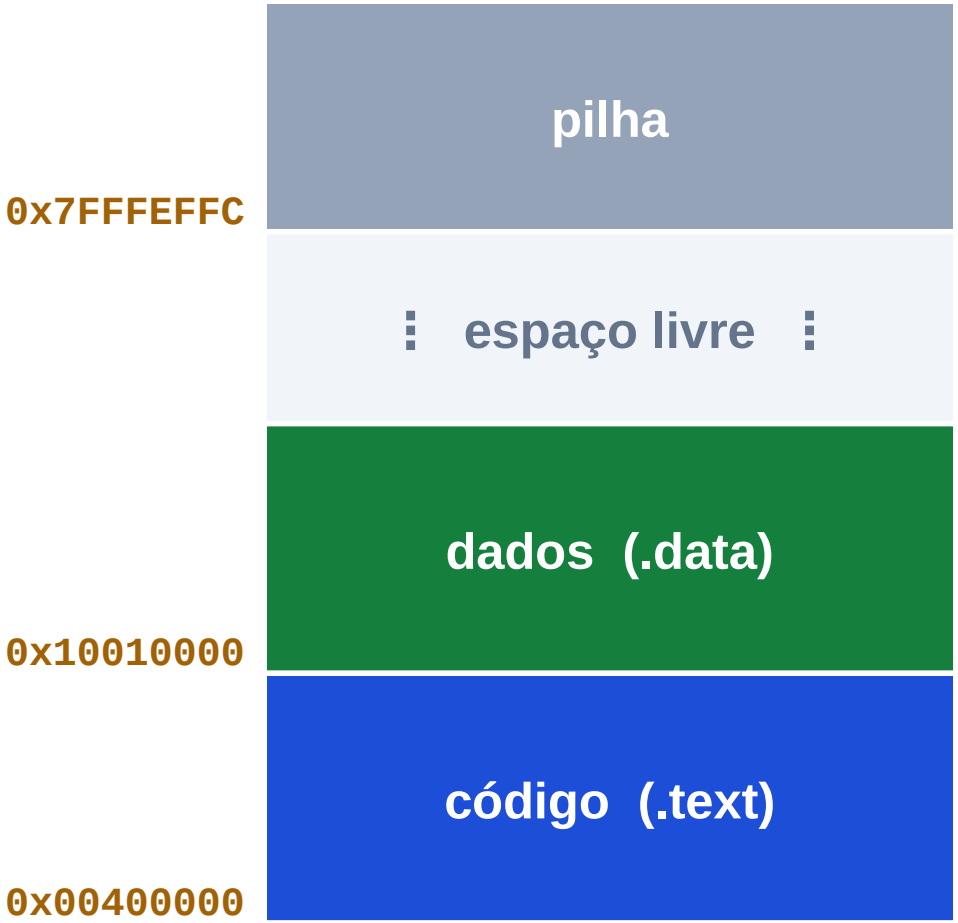


Endereço O número da prateleira. Diz ONDE.

Valor O que está em cima da prateleira. Diz O QUÊ.

Um registrador guarda 32 bits. Se esses bits são um valor ou um endereço, quem decide é o programa — não o circuito.

A memória é dividida em regiões



cresce para baixo — encontro 5

vetores, variáveis, textos

as instruções do programa

`.text` e `.data` são recados para o montador: “o que vem a seguir vai para esta região”.

No encontro 1, o programa só tinha .text. Hoje ele ganha uma região de dados.

O que o montador faz com o .data

```
        .data
vetor:  .word  7, 3, 9, 2, 5
soma:   .word  0
msg:    .asciiz "Soma = "
```

Rótulo	Vira o endereço	Conteúdo
`vetor`	`0x10010000`	7, 3, 9, 2, 5 (5 palavras)
`soma`	`0x10010014`	0 (1 palavra)
`msg`	`0x10010018`	os caracteres do texto

`.word` reserva palavras de 32 bits, já com esses valores

`.asciiz` guarda um texto, com um byte zero no fim ("z" de zero)

`rótulo:` dá nome ao endereço onde o dado começou — como main: no encontro 1

soma começa 20 bytes depois de vetor: 5 palavras × 4 bytes. O montador empilha os dados um atrás do outro, na ordem em que foram escritos.

8 · la — LOAD ADDRESS

la coloca um endereço num registrador

```
la   $s0, vetor
```

o montador transforma em:

```
lui $1, 0x1001       # $at = 0x10010000
ori $16, $1, 0       # $s0 = $at OU 0
```

Mais uma pseudoinstrução que vira duas — e o **$at** de novo como rascunho, como no li $s1, 100000 do encontro 2.

Painel de registradores, depois do la

$s0 **268500992**

em hexadecimal: **0x10010000**

Não é 7. É onde o 7 está.

Um número estranhamente grande num registrador quase sempre é um endereço.

Se o MARS mostrar lui $1,4097: é o mesmo número, em decimal (0x1001 = 4097).

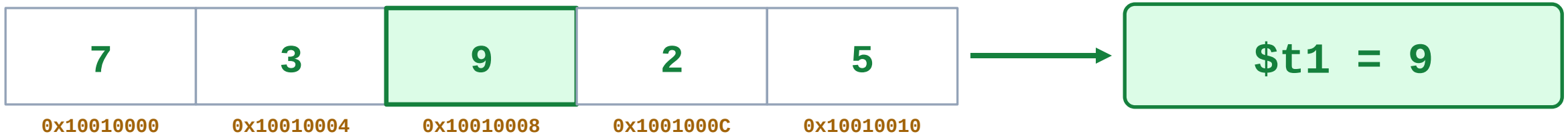
lw busca uma palavra no almoxarifado

```
lw  $t1, 8($s0)
```

$t1 ← Mem[$s0 + 8]

base	$s0 = 0x10010000
deslocamento	+ 8
endereço	= 0x10010008

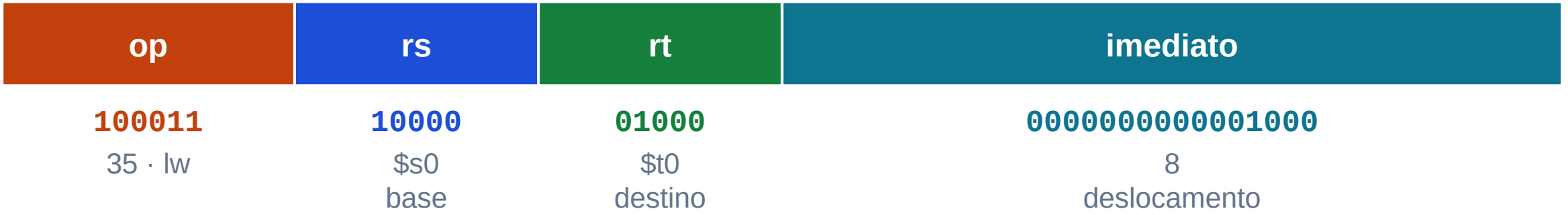
Por que 8 e não 2? O deslocamento é em bytes. vetor[2] está 2 palavras adiante = 8 bytes.



A memória não muda: ler não apaga — igual aos registradores.

O imediato virou um pedaço de endereço

```
lw  $t0, 8($s0)      →      0x8E080008
```



É formato I, como vocês já sabiam. rs guarda a base, rt o destino, e o imediato de 16 bits é somado à base para formar o endereço — com extensão de sinal: pode ser negativo.

O circuito que soma base + deslocamento é a mesma ULA que faz o **add**. Uma instrução de memória também é, antes de tudo, uma conta.

sw devolve ao almoxarifado — e a direção inverte

```
sw  $t2, 20($s0)          Mem[ $s0 + 20 ]  ←  $t2
```

`lw  $t1, 8($s0)`	**registrador ← memória**	o primeiro registrador é DESTINO
`sw  $t2, 20($s0)`	**registrador → memória**	o primeiro registrador é ORIGEM

A exceção prometida no encontro 2. Até aqui, “o primeiro registrador é o destino” valia sempre. No sw, não: nenhum registrador é escrito. Quem muda é a memória. Nos bits, o $t2 continua no campo rt — que, aqui, é origem.

```
sw  $t2, 20($s0)   →   0xAE0A0014
```

vetor[i] mora em base + 4 · i

Em C	i	Deslocamento 4 · i	Endereço	Valor	Em assembly
`vetor[0]`	0	**0**	**0x10010000**	**7**	`lw $t1, 0($s0)`
`vetor[1]`	1	**4**	**0x10010004**	**3**	`lw $t1, 4($s0)`
`vetor[2]`	2	**8**	**0x10010008**	**9**	`lw $t1, 8($s0)`
`vetor[3]`	3	**12**	**0x1001000C**	**2**	`lw $t1, 12($s0)`
`vetor[4]`	4	**16**	**0x10010010**	**5**	`lw $t1, 16($s0)`

Em C, o índice conta elementos. Em assembly, o deslocamento conta bytes. Quem faz a conversão — multiplicar por 4 — é você. Em C, o compilador faz por você.

É por isso que vetores em C começam no índice 0: vetor[0] está exatamente no endereço-base.

Somando o vetor: buscar, somar, buscar, somar…

```
la   $s0, vetor      # base
lw   $t0, 0($s0)     # 7
lw   $t1, 4($s0)     # 3
add  $t2, $t0, $t1
lw   $t1, 8($s0)     # 9
add  $t2, $t2, $t1
lw   $t1, 12($s0)    # 2
add  $t2, $t2, $t1
lw   $t1, 16($s0)    # 5
add  $t2, $t2, $t1
sw   $t2, 20($s0)    # soma = 26
```

$t2 ao longo do programa

7 + 3	→	**10**
+ 9	→	**19**
+ 2	→	**21**
+ 5	→	**26**

Repare no padrão: o mesmo par lw + add, copiado e colado. Só o deslocamento muda: 4, 8, 12, 16. Com mil elementos, seriam mil pares.

O programa não sabe falar com a tela. Ele pede.

Uma chamada de sistema tem sempre a mesma forma:

1. **Qual serviço?** `número em $v0`
2. **Com que dado?** `argumento em $a0`
3. **Executa** `syscall`

$v0	Serviço	Espera / devolve
1	imprimir inteiro	**$a0 = o VALOR**
4	imprimir texto	**$a0 = o ENDEREÇO do texto**
5	ler inteiro	o número lido chega em $v0
10	encerrar	—
11	imprimir caractere	$a0 = o código do caractere

Mistério do encontro 1, resolvido: li $v0, 10 + syscall = “serviço 10, por favor”. Era um pedido de encerramento o tempo todo.

Fonte: Patterson & Hennessy, Apêndice A, Figura A.9.1 — os códigos valem no MARS.

Texto quer endereço. Número quer valor.

imprimir o texto

```
li   $v0, 4
la   $a0, msg
syscall
```

la: $a0 recebe ONDE o texto começa. O serviço vai lendo byte a byte até achar o zero.

imprimir o número

```
li   $v0, 1
lw   $a0, 20($s0)
syscall
```

lw: $a0 recebe o próprio número guardado em soma.

Run I/O

Soma = 26

Pela primeira vez no curso, o programa mostra alguma coisa.

Prevejam antes de executar

```
        .data
v:      .word  4, 8, 15, 16
        .text
main:   la   $t0, v
        lw   $t1, 12($t0)
        lw   $t2, 4($t0)
        sub  $t3, $t1, $t2
        sw   $t3, 0($t0)
```



1. Quanto vale $t3?
2. Como fica a memória de v no final?
3. E se o primeiro lw fosse lw $t1, 6($t0)?

lê 8 · lê 15 · erro

Atividade

1	**Observar** — montar o aula3_vetor.asm e achar o vetor na janela Data Segment	8 min
2	**Prever** — escrever o que vai sair na tela e acompanhar a memória mudar	8 min
3	**Modificar** — trocar um valor do vetor e um deslocamento — e ver o que quebra	8 min
4	**Provocar** — trocar lw por la na impressão e explicar o número que aparecer	6 min

O enunciado está na folha de atividade. A tabela de chamadas de sistema está na folha de consulta, na última página.

Duas surpresas

`lw  $t1, 10($s0)`

```
Runtime exception at 0x00400014:
fetch address not aligned
on word boundary 0x1001000a
```

0x1001000A não é múltiplo de 4: fica no meio de duas palavras. O MIPS não lê palavra desalinhada.

`la  $a0, soma`

```
Run I/O:

Soma = 268501012
```

268 501 012 = **`0x10010014`**: o endereço de soma. O serviço 1 imprime o que estiver em $a0 — sem saber se é valor ou endereço.

O processador não confere se 32 bits são um valor ou um endereço. Essa responsabilidade é de quem escreve o programa.

O que fica de hoje

1. A memória é uma fila de bytes numerados. O número é o endereço.
2. Uma palavra ocupa 4 bytes e começa num endereço múltiplo de 4.
3. Valor e endereço são coisas diferentes — mesmo quando os dois estão num registrador.
4. lw traz da memória; sw leva para a memória. No sw, o registrador é origem.
5. Para mostrar ou ler algo, o programa pede ao sistema: $v0, $a0, syscall.

Âmbar é onde. Verde é o quê.

20 · SEMANA QUE VEM

Dois problemas que ficaram sem solução

Repetição

Cinco elementos, cinco pares lw + add copiados. Com mil elementos, mil pares?

Decisão

Qual é o maior elemento do vetor? Para saber, é preciso comparar — e fazer coisas diferentes conforme a resposta.

`beq` `bne` `slt` `j`

Quatro instruções resolvem os dois — e com elas dá para escrever qualquer programa.

Na aula 4: controle de fluxo.